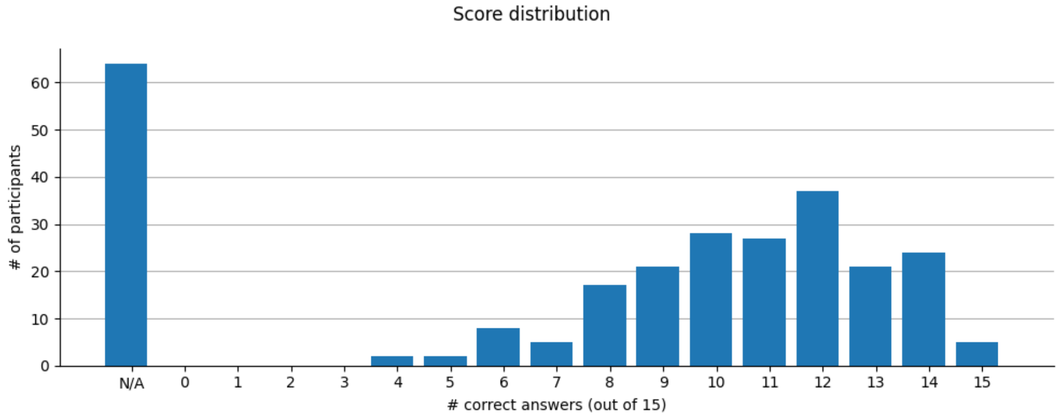
CIS*3110 Operating Systems

Week 3
Lecture #4 Virtualization: Scheduling

Prof. Yulia Kotseruba
January 20, 2026

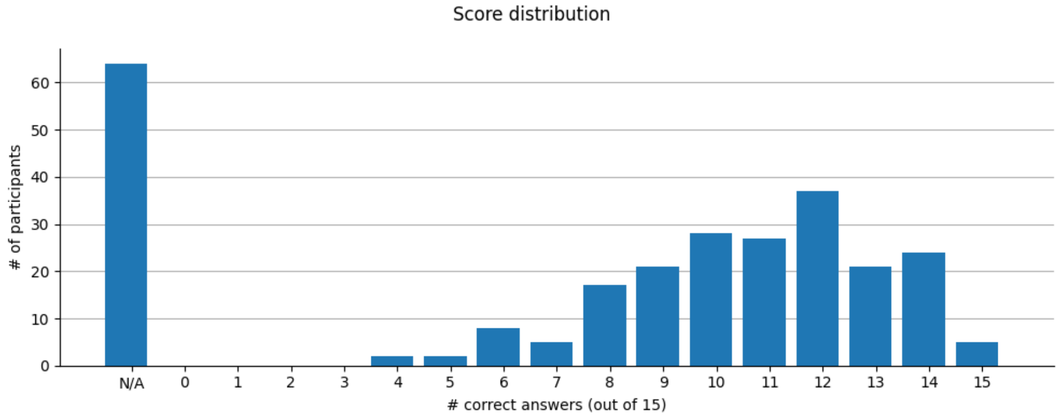
Quiz 2

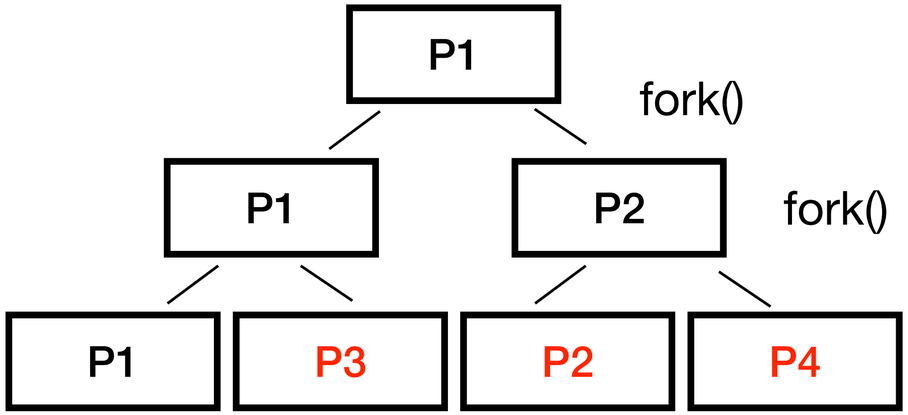
- **71%** average (down from 75% quiz 1)
- 3 hardest questions:
  - (42%) How many NEW processes will be created by TWO consecutive successful `fork()` calls a) 2 b) 3 c) 5 d) 8
  - (45%) What is inherited by a child from a parent after a successful call to `fork()`? a) address space b) parent's threads c) signals d) all of the above
  - (51%) What is NOT a primary state of a process a) running b) ready c) sleeping d) blocked



Quiz 2

- **71%** average (down from 75% last week)
- 3 hardest questions:
  - (42%) How many NEW processes will be created by TWO consecutive successful `fork()` calls a) 2 b) 3 c) 5 d) 8
  - (45%) What is inherited by a child from a parent after a successful call to `fork()`? a) address space b) parent's threads c) signals d) all of the above
  - (51%) What is NOT a primary state of a process a) running b) ready c) sleeping d) blocked





4 processes total, 3 NEW

Recap

- Processes are **running programs** managed by an OS
- Each process has a **state**: address space (memory), CPU registers (PC, stack pointer, etc), and I/O status (open files, etc.)
- OS has **data structures** (PCB) and **system calls** to manage state of the processes
- `fork()`, `exec()`, `wait()` and `kill()` are the key **system calls** that OS exposes to create, modify, suspend, and terminate processes
- `ps` and (h)`top` are useful **utilities** for monitoring processes

Recap

- **Virtualization** - turn a physical resource into a virtual form of itself
- Allows many programs to **run concurrently** on the same CPU
  - cpu.c example from Lecture 2
  - fork() from Lecture 3

Recap

- Recall Lecture 2 about how OS runs the programs
- When a user runs a program via terminal or UI **(OS manages I/O)**
  - Program is loaded into memory **(OS loader)**
  - Executed line by line in the CPU **(OS fetch-decode-execute cycle)**
  - Upon completion, the results are returned to user/saved/discarded **(OS reclaims resources and manages I/O if needed)**

Direct execution

- Direct execution - running user programs directly on a CPU
- **Problem #1** What if a user program does something bad? e.g. overwrites part of the OS itself, initiates a super-long I/O operation
- **Problem #2** Once a process occupies the CPU, how can it be switched to a different process (e.g. to the OS itself)?
- **Problem #3** If there are multiple processes, which process should the CPU switch to?

Kernel and user mode

- CPU can run in **two modes***
  - **User mode** - restricted in what it can do (e.g. cannot write directly to disk, does not have full access to hardware)
  - **Kernel mode** - privileged mode in which the OS kernel runs
- Switching between the modes:
  - **Traps (interrupts)** - jump into the OS kernel and raise the privilege level
  - **Return-from-trap** - is called at the end of the kernel instruction, returns into the user program and reduces the privilege level
  - **Trap table** - a table that maps system calls to kernel functions

*some OSs have more, but these are two basic ones

Kernel and user mode

- OS switches the CPU between **kernel** and **user** modes
- **User** code can **call system calls** through API provided by the operating system
- OS **kernel** is the **collection of routines**
  - implements system calls
  - runs on computer hardware to serve users' programs
- **Kernel code** is executed only in **kernel mode**

System calls

- Programmer **interface** for the services provided by an **operating system**
- Typically functions written in C (and some in assembly)
- **6 types** of system calls

- **process control** - creation, deletion, and management of processes `fork(), wait()`
- **file management** - creation, deletion, and management of files `open(), close(), write()`
- **device management** - requesting/releasing devices, device attributes `read(), write()`
- **information maintenance** - get time/date, system data `getpid()`
- **communications** - send/receive messages, transfer data `pipe()`
- **protection** - get/set file permissions `chmod(), chown()`

Interrupts

- OS switches between kernel and user modes through interrupts
- **Hardware** interrupts (asynchrononous)
  - I/O interrupts - caused by I/O completion and malfunctioning of I/O devices
  - Timer interrupt - raised at fixed intervals or when a specified interval elapses (e.g. sleep() function call)
- **Software** interrupts - traps, faults, exceptions (synchronous)
  - Exceptional conditions during execution of an instruction (e.g. overflow, memory protection violation, etc)
  - Calling the software interrupt instruction

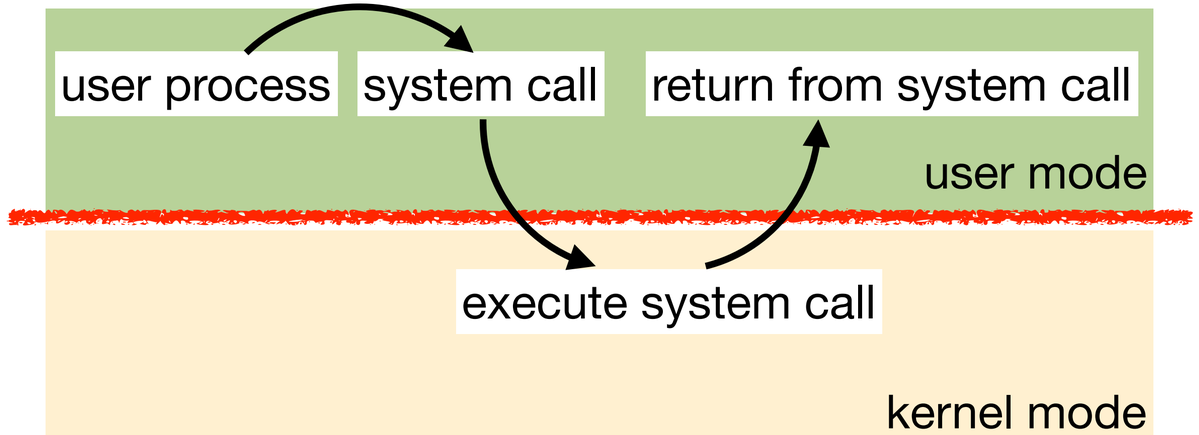
Kernel and user mode

```
#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <fcntl.h>
#include <sys/types.h>

int main(int argc, char *argv[]) {
    int fd = open("/tmp/file", O_WRONLY|
    O_CREAT|O_TRUNC, S_IRWXU);

    assert(fd > -1);

    int rc = write(fd, "hello world\n", 13);
    assert(rc == 13);
    close(fd);
    return 0;
}
```

`open()`, `write()`, and `close()` are called in user mode

OS will switch the CPU to kernel mode to execute these functions

OS will then switch the CPU back to user mode



Kernel and user mode

Steps of calling write(fd, buffer, nbytes)

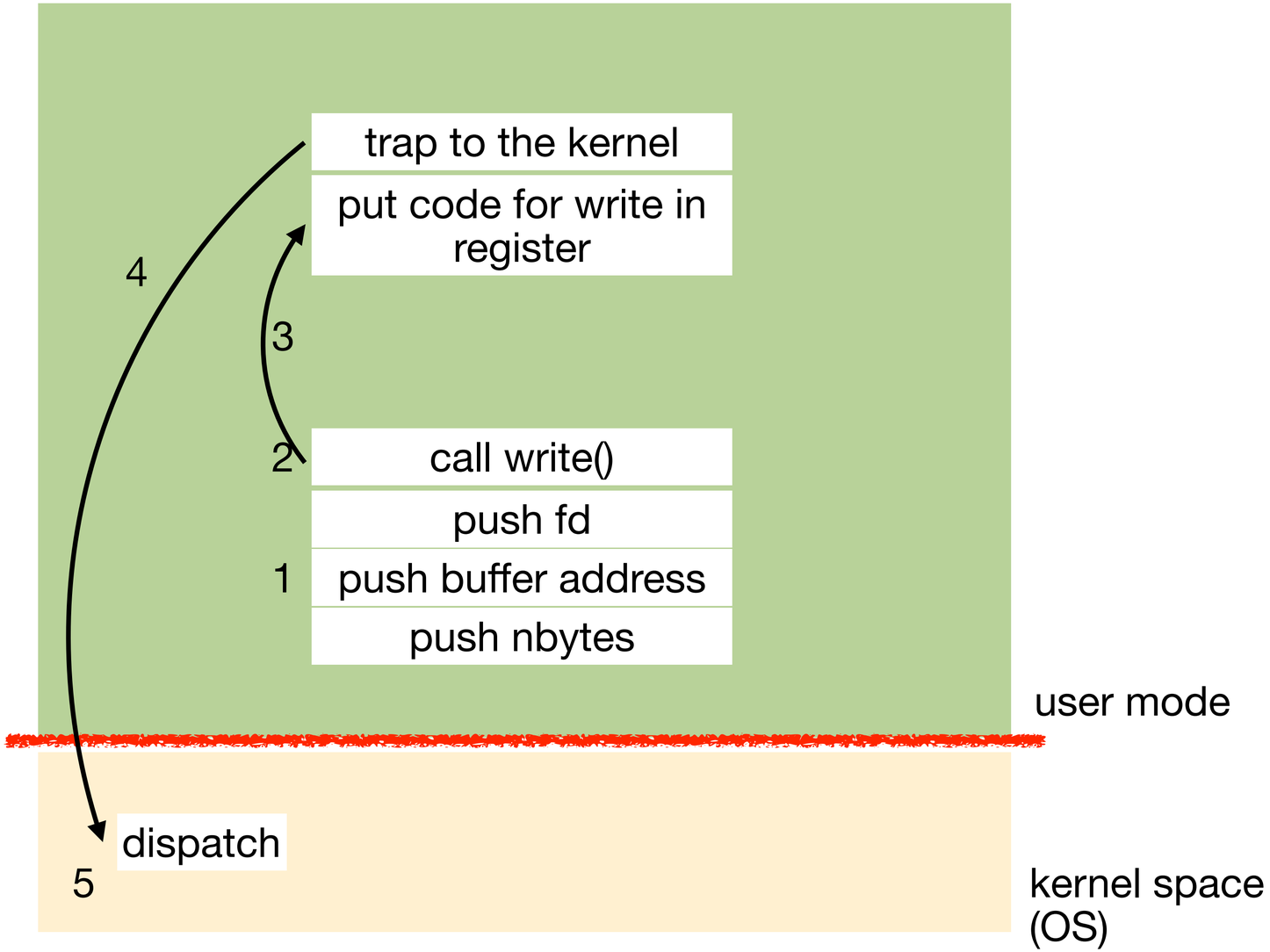
1. Push `write()` params onto stack

2. Call `write()` system call

3. Write calls a library which puts system call number for `write()` in a special register

4. Execute a trap instruction to switch to kernel mode

5. Start execution at a fixed address within the kernel



Kernel and user mode

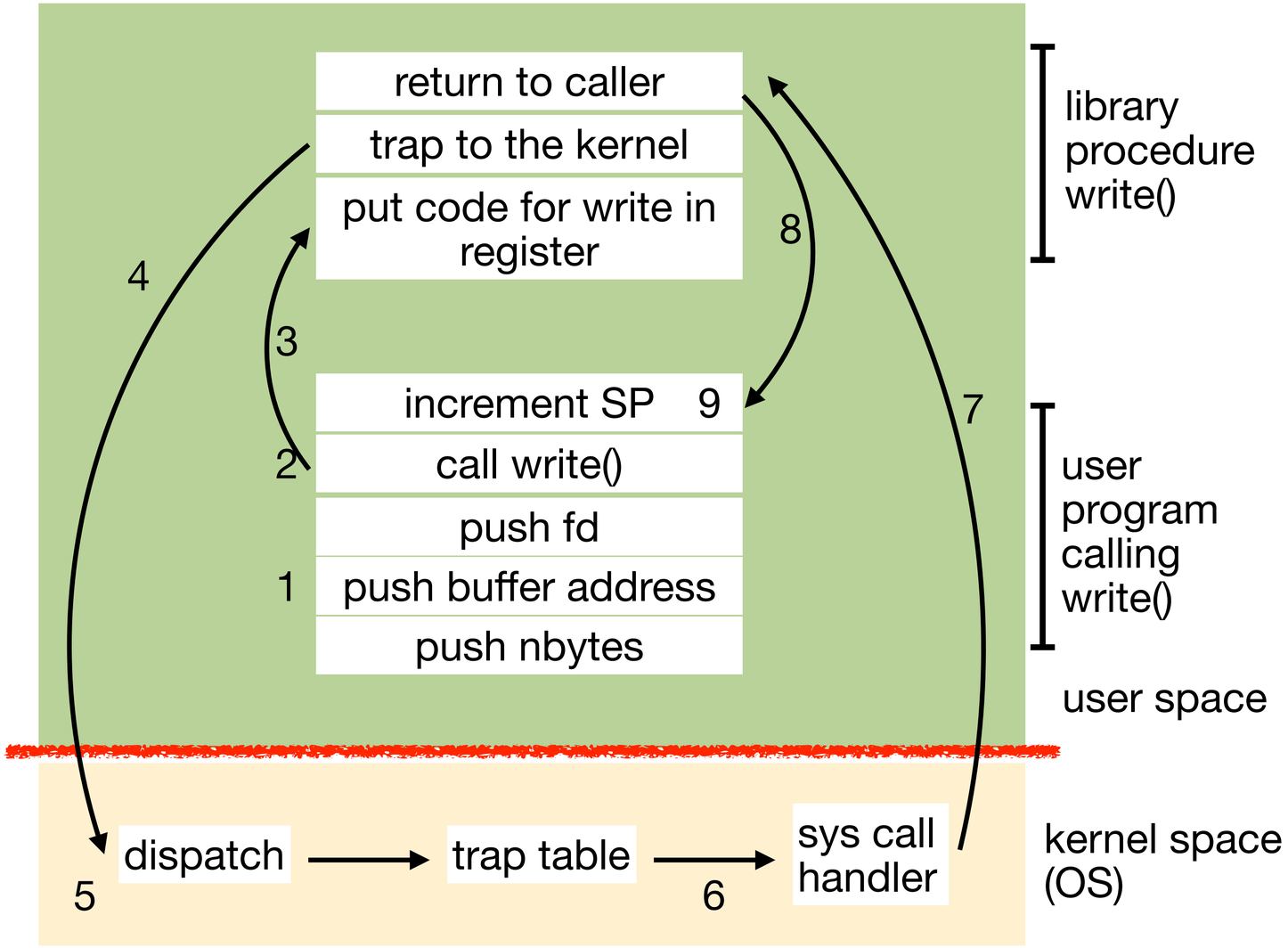
Steps of calling write(fd, buffer, nbytes)

6. Kernel checks the number of the system call in the trap table and calls the system call handler

7. Once system call handler completes, it returns control to the library procedure following the trap instruction

8. The library procedure returns to the user program

9. User program continues execution



Direct execution

- **Problem #1** What if the program does something bad? e.g. overwrites part of the OS itself, initiates a super-long I/O operation
  - **Solution**
    - limit direct execution only to OS processes (kernel mode)
    - run user programs in user mode
    - switch from user to kernel mode via interrupts (traps)

- **Problem #2** Once the process occupies the CPU, how can it be switched to a different process?

Context switch

- **Cooperative approach**
  - OS waits for process to finish/fail and return control
- **Non-cooperative approach**
  - OS periodically raises a timed interrupt and takes control
- Once in control, OS performs a **context switch**
  - saves registers and stack pointer of the running process
  - restores registers and stack pointer for a different process
  - the process is nearly the same regardless of whether processes run in privileged or user mode

Direct execution

- **Problem #1** What if the program does something bad? e.g. overwrites part of the OS itself, initiates a super-long I/O operation
  - Solution
    - limit direct execution only to OS processes (kernel mode)
    - run user programs in user mode
    - switch from user to kernel mode via interrupts (traps)

- **Problem #2** Once the process occupies the CPU, how can it be switched to a different process?
  - Solution: wait for system calls or issue timed interrupts

- **Problem #3** If there are multiple processes, which process should the CPU switch to?

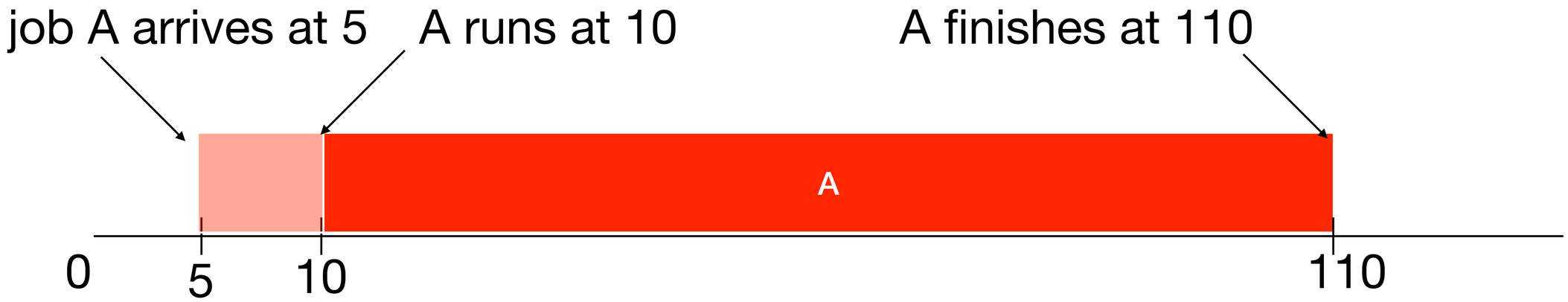
  - Solution: scheduling

Scheduling

- Scheduling **policy** - how to choose the next process
- Scheduling **metrics** - how to compare the policies
  - **Turnaround time** - amount of time it takes for a process to finish after arrival
  - **Response time** - how soon the job is scheduled to run after arrival

Scheduling

- Scheduling **policy** - how to choose the next process
- Scheduling **metrics** - how to compare the policies
  - **Turnaround time** - amount of time it takes for a process to finish after arrival
  - **Response time** - how soon the job is scheduled to run after arrival



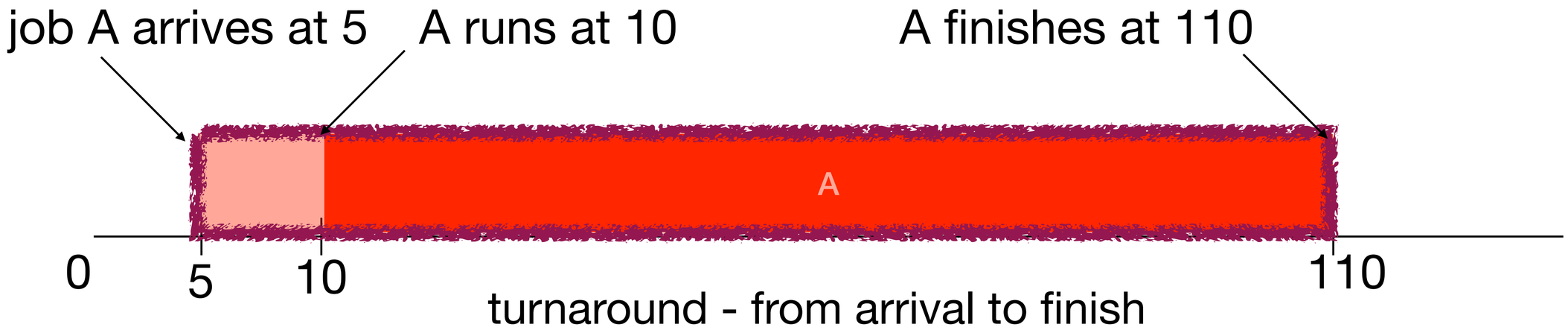
Scheduling

- Scheduling **policy** - how to choose the next process
- Scheduling **metrics** - how to compare the policies
  - **Turnaround time** - amount of time it takes for a process to finish after arrival
  - **Response time** - how soon the job is scheduled to run after arrival

response time - from arrival to start

Scheduling

- Scheduling **policy** - how to choose the next process
- Scheduling **metrics** - how to compare the policies
  - **Turnaround time** - amount of time it takes for a process to finish after arrival
  - **Response time** - how soon the job is scheduled to run after arrival



Assumptions

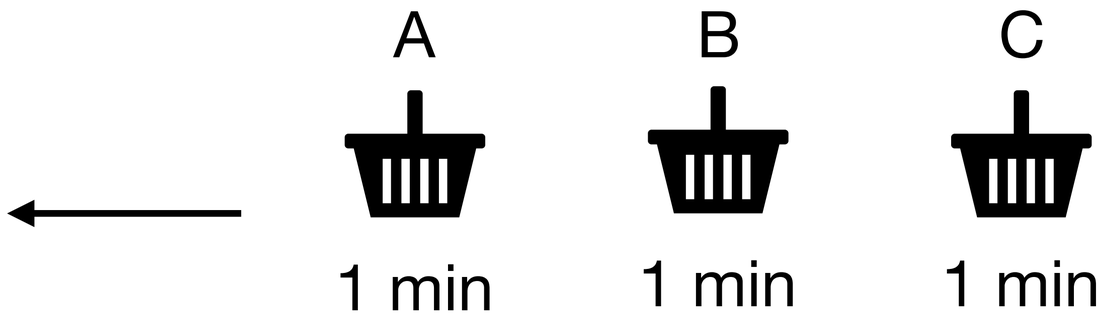
- Processes (jobs)
  - Are equal in duration (CPU burst)
  - Arrive at the same time
  - Run to completion (no preemption, no termination, no infinite loops)
  - Run on CPU only (no I/O)
  - Have a known run-time

First in, first out (FIFO)

- The simplest scheduling, aka "**first come, first served**" (FCFS)
- Jobs are added to the queue as they arrive
- The **first job** in the queue runs until it **finishes**
- Then the next job starts
- etc..

First in, first out (FIFO)

3 customers arrive at the cash register at the same time (t=0) and line up



It takes a cashier 1 min to serve each customer

What is the turnaround time for each customer?

$$T_{turnaround} = T_{completion} - T_{arrival}$$

A $T_{turnaround} = 1 - 0 = 1$

B $T_{turnaround} = 2 - 0 = 2$

C $T_{turnaround} = 3 - 0 = 3$

What is the average turnaround?

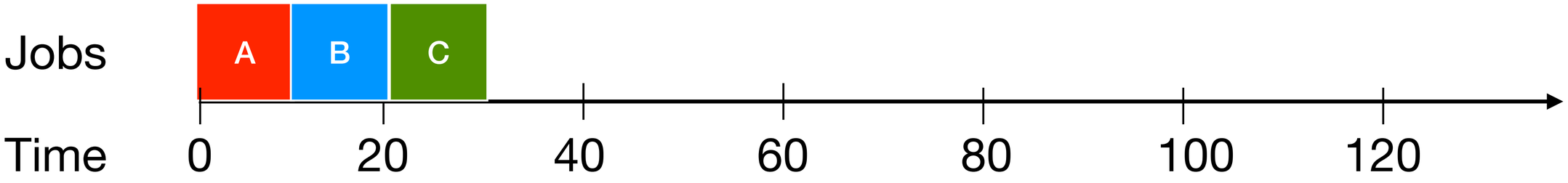
$$Avg_T_{turnaround} = \frac{1+2+3}{3} = 2$$

First in, first out (FIFO)

CPU burst - when the program actively uses CPU

A=B=C=10s

A, B, C arrive at t=0



What is the turnaround time for each job?

$$T_{turnaround} = T_{completion} - T_{arrival}$$

A $\quad T_{turnaround} = 10 - 0 = 10$

B $\quad T_{turnaround} = 20 - 0 = 20$

C $\quad T_{turnaround} = 30 - 0 = 30$

What is the average turnaround?

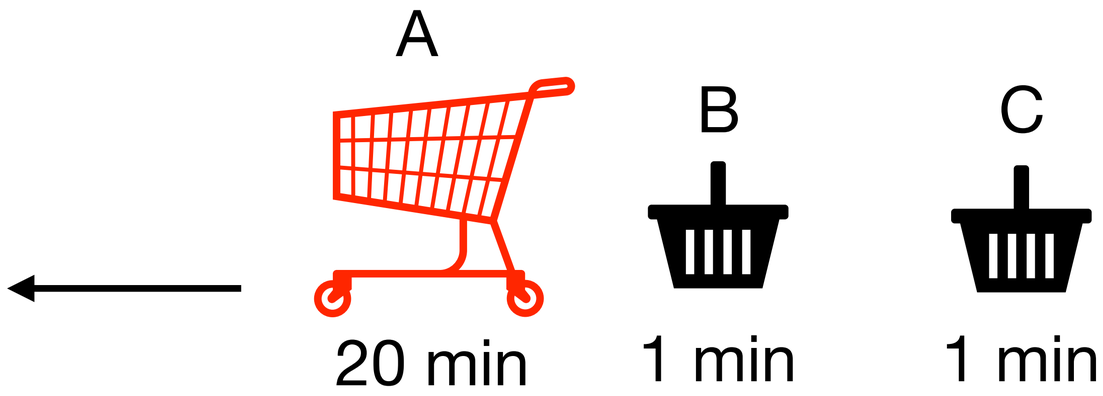
$$Avg_T_{turnaround} = \frac{10 + 20 + 30}{3} = 20$$

Assumptions

- Processes
  - ~~Are equal in duration~~
  - Arrive at the same time
  - Run to completion
  - Run on CPU only (no I/O)
  - Have a known run-time

First in, first out (FIFO)

3 customers arrive at the cash register at the same time (t=0) and line up



1 customer takes 20 min, the other two are still 1 min

What is the turnaround time for each customer?

$$T_{turnaround} = T_{completion} - T_{arrival}$$

A $T_{turnaround} = 20 - 0 = 20$

B $T_{turnaround} = 21 - 0 = 21$

C $T_{turnaround} = 22 - 0 = 22$

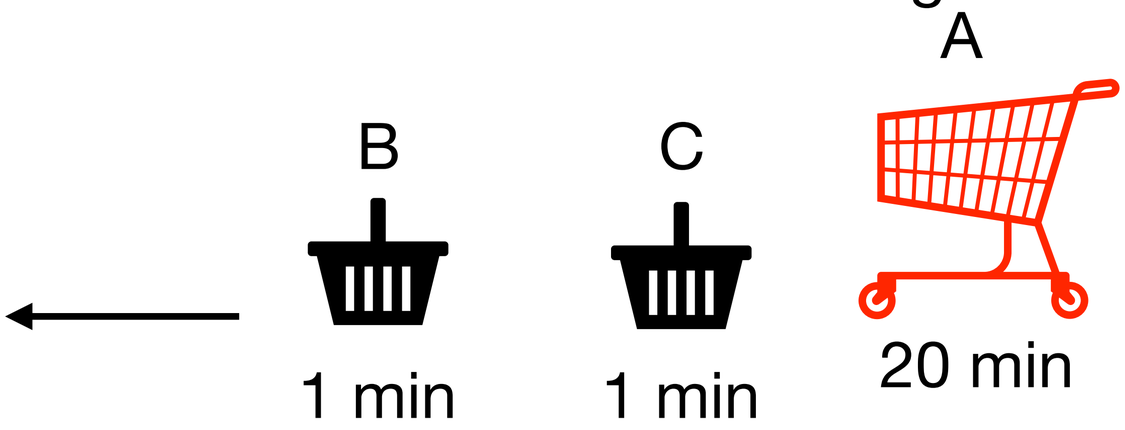
What is the average turnaround?

$$Avg_T_{turnaround} = \frac{20 + 21 + 22}{3} = 21$$

convoy effect - when one heavy process delays many short ones

Shortest Job First (SJF)

3 customers arrive at the cash register at the same time (t=0) and line up



1 customer takes 20 min, the other two are still 1 min

process the shortest ones first

What is the turnaround time for each customer?

$$T_{turnaround} = T_{completion} - T_{arrival}$$

B $\quad T_{turnaround} = 1 - 0 = 1$

C $\quad T_{turnaround} = 2 - 0 = 2$

A $\quad T_{turnaround} = 22 - 0 = 22$

What is the average turnaround?

$$Avg_T_{turnaround} = \frac{22 + 1 + 2}{3} = 8.33$$

order matters, when A was first, avg turnaround was 21 m

Shortest Job First (SJF)

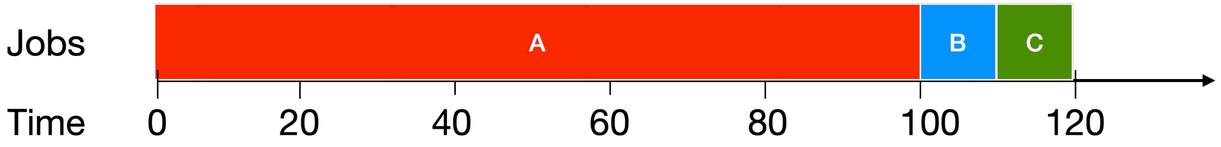
- Jobs are added to the queue as they arrive
- **The shortest job** is selected and runs until **completion**
- Then the **next shortest job**
- etc..

FIFO vs. SJF

What happens if jobs have different runtimes?

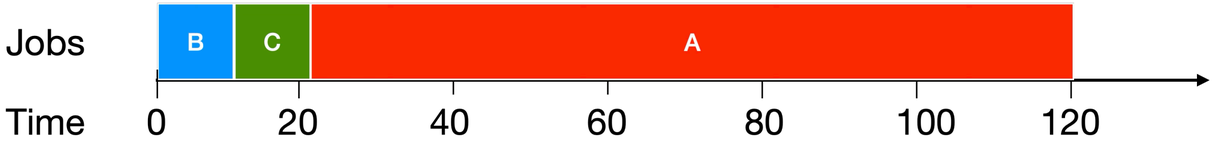
A=100s, B=C=10s

FIFO



$$Avg_T_{turnaround} = \frac{100 + 110 + 120}{3} = 110$$

SJF



$$Avg_T_{turnaround} = \frac{10 + 20 + 100}{3} = 43.33$$

SJF is > 2x better

Assumptions

- Processes
  - ~~Are equal in duration~~
  - Arrive at the same time
  - Run to completion
  - Run on CPU only (no I/O)
  - Have a known run-time

SJF is optimal under these assumptions

i.e. will always give the best average turnaround time

SJF was used in the early batch systems. All programs were known in advance, the algorithm just needed to give the results back in an optimal time for all users

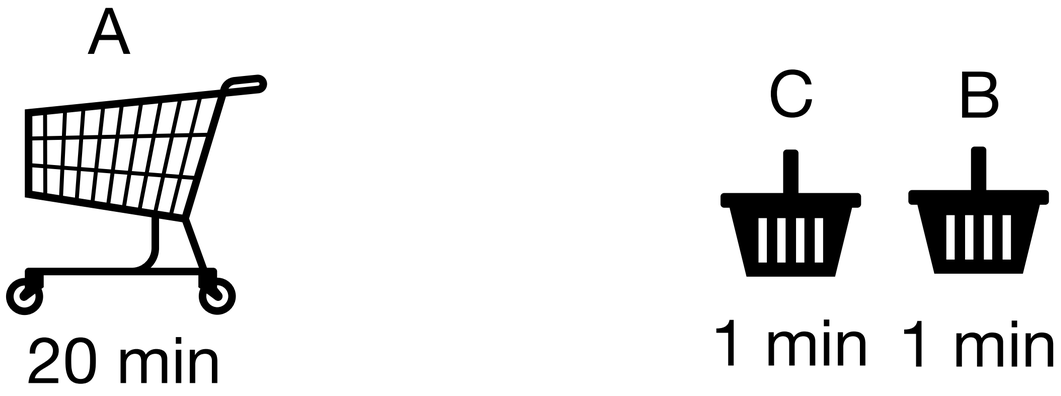
Assumptions

- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - Run to completion
  - Run on CPU only (no I/O)
  - Have a known run-time

Is SJF still optimal if jobs do not arrive at the same time? NO

when C and B show up, A is already being processed

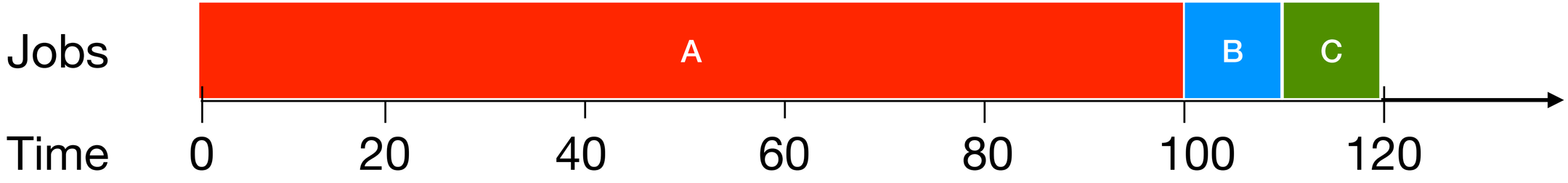
we are back to where we started



SJF (late arrivals)

What happens if jobs have **different runtimes** and arrive at **different times**?

Duration: A=100s, B=C=10s



A arrives at 0 s $T_{turnaround} = 100 - 0 = 100$

B arrives at 10 s $T_{turnaround} = 110 - 10 = 100$

C arrives at 10 s $T_{turnaround} = 120 - 10 = 110$

$$Avg_T_{turnaround} = \frac{100 + 100 + 110}{3} = 103.33$$

What is FIFO avg turnaround in this case? Same

Assumptions

- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - ~~Run to completion~~
  - Run on CPU only (no I/O)
  - Have a known run-time

What if we don’t need to wait for jobs to finish?

Assumptions

- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - ~~Run to completion~~
  - Run on CPU only (no I/O)
  - Have a known run-time

What if we don’t need to wait for jobs to finish?

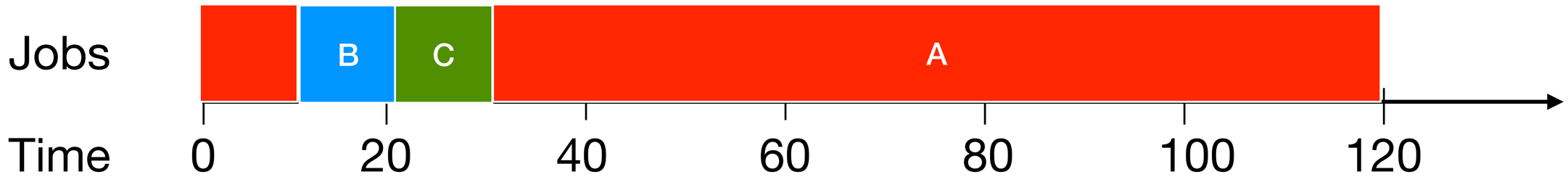
Cashier starts serving customer A, but when C and B arrive, switches to serving them, and then goes back to A

Preemptive Shortest Job First (PSJF)

- aka **Shortest Time-to-Completion First** (STCF)
- Jobs are added to the queue as they arrive
- Any time a new job is added
- The one with **shortest time to completion** is scheduled
  - If a new job will finish sooner than current, current job will be **pre-empted**
  - If a current job will finish sooner than new one, current job **continues**
- etc..

PSJF (late arrivals)

Duration: A=100s, B=C=10s Arrival: A=0s, B=C=10s



A=100s arrives at t=0

B=C=10s arrive at t=10

A is pre-empted at t=10

B and C run at t=10 and t=20

A continues at t=30

What are the turnaround times?

A arrives at 0 s $T_{turnaround} = 120 - 0 = 120$

B arrives at 10 s $T_{turnaround} = 20 - 10 = 10$

C arrives at 10 s $T_{turnaround} = 30 - 10 = 20$

$$Avg_T_{turnaround} = \frac{120 + 10 + 20}{3} = 50$$

Assumptions

- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - ~~Run to completion~~
  - Run on CPU only (no I/O)
  - Have a known run-time

Under these assumptions, PSJF gives provably optimal avg turnaround time

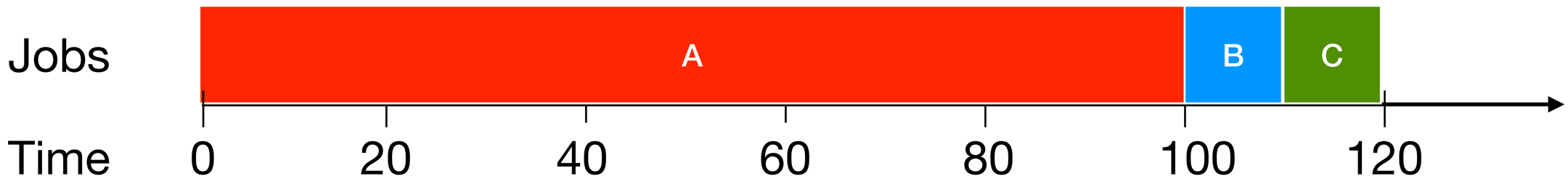
What about **response time**?

$$T_{response} = T_{first\ run} - T_{arrival}$$

Time from when the job arrives to the first time it is scheduled

SJF

Duration: A=100s, B=C=10s Arrival: A=0s, B=C=10s



What is the response time for each job?

$$T_{response} = T_{first\ run} - T_{arrival}$$

A arrives at 0 s $T_{response} = 0 - 0 = 0$

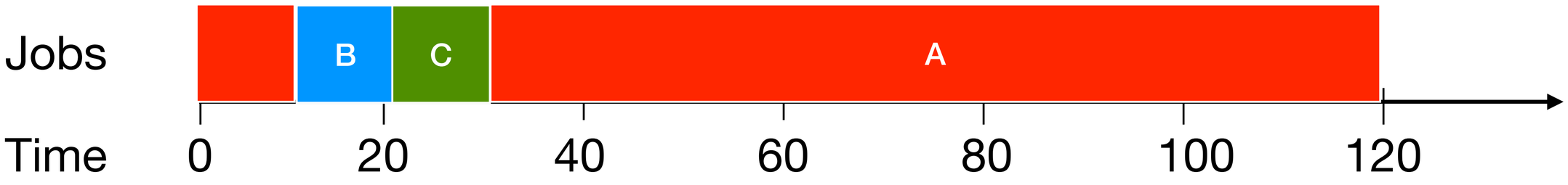
B arrives at 10 s $T_{response} = 100 - 10 = 90$

C arrives at 10 s $T_{response} = 110 - 10 = 100$

$$Avg_T_{response} = \frac{190}{3} = 63.33$$

PSJF (late arrivals)

Duration: A=100s, B=C=10s Arrival: A=0s, B=C=10s



What is the response time for each job?

$$T_{response} = T_{first\ run} - T_{arrival}$$

$$T_{response} = 0 - 0 = 0$$

$$T_{response} = 10 - 10 = 0$$

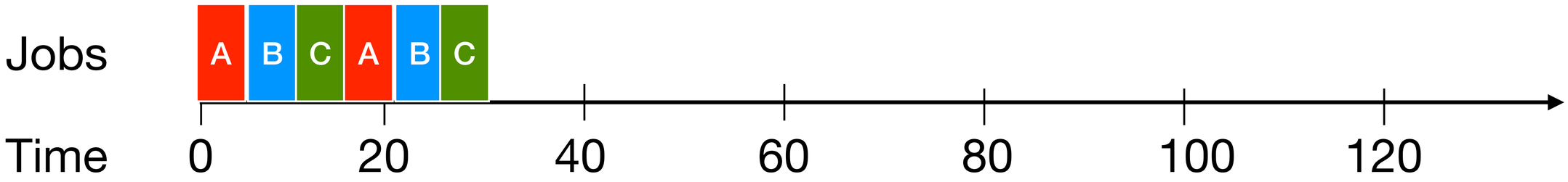
$$T_{response} = 20 - 10 = 10$$

$$Avg_T_{response} = \frac{10}{3} = 3.33$$

pre-emption improves response by 10x (SJF was 63.3)

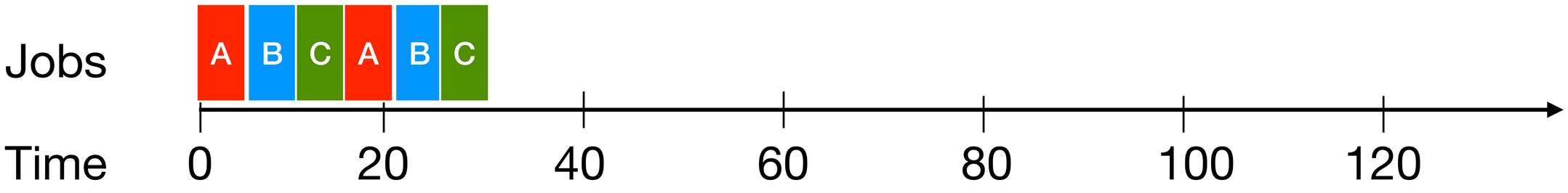
Round Robin

- Jobs are added to the queue as they arrive
- Run the first job in the queue for a time slice (quantum) or until the job is done
- Then switch to the next job and run for a time slice (quantum) or until the job is done
- etc..



Round Robin

Duration: A=B=C=10s Arrival: A=B=C=0 Quantum=5



What is the response time for each job?

$$T_{response} = T_{first\ run} - T_{arrival}$$

A $T_{response} = 0 - 0 = 0$

B $T_{response} = 5 - 0 = 5$

C $T_{response} = 10 - 0 = 10$

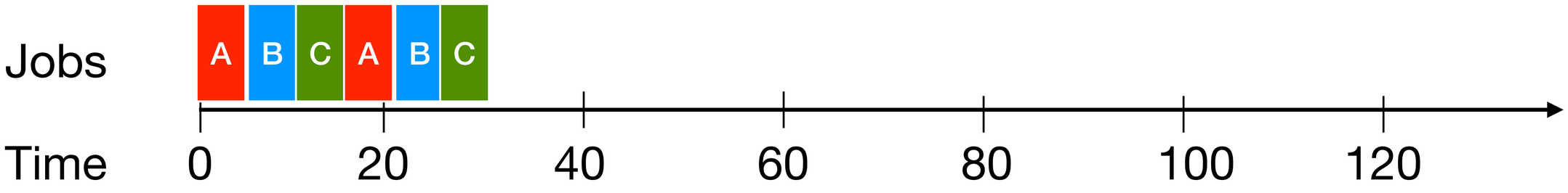
$$Avg_T_{response} = \frac{15}{3} = 5$$

What is avg response for SJF in the same scenario?

$$Avg_T_{response} = \frac{0 + 10 + 20}{3} = 10$$

Round Robin

Duration: A=B=C=10s Arrival: A=B=C=0 Quantum=5



What is the turnaround time for each job?

A $T_{turnaround} = 20 - 0 = 20$

B $T_{turnaround} = 25 - 0 = 25$

C $T_{turnaround} = 30 - 0 = 30$

$$Avg_T_{turnaround} = \frac{20 + 25 + 30}{3} = 25$$

What is avg turnaround for SJF in the same scenario?

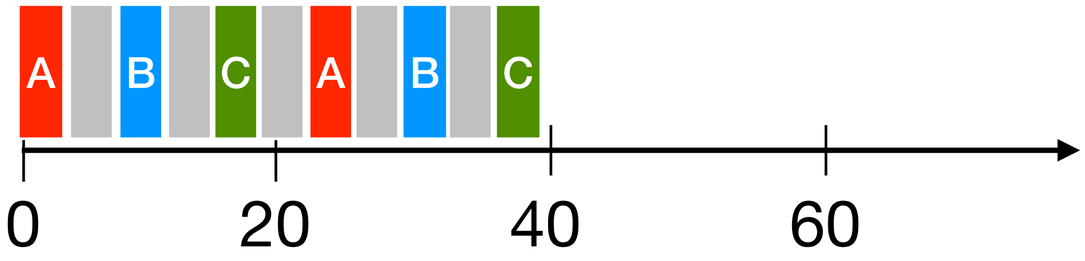
$$Avg_T_{turnaround} = \frac{10 + 20 + 30}{3} = 20$$

Assumptions

- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - ~~Run to completion~~
  - Run on CPU only (no I/O)
  - Have a known run-time

Round robin is the best in terms of response time so far but at the expense of turnaround

Switching between processes is not free, therefore very small quantums should be avoided (context



How does round robin with a very large quantum behave? Degrades to FIFO

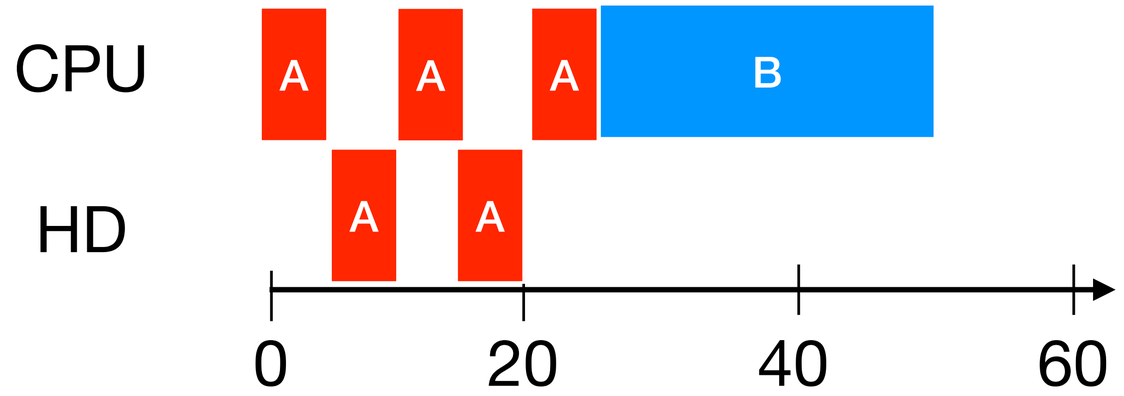
Assumptions

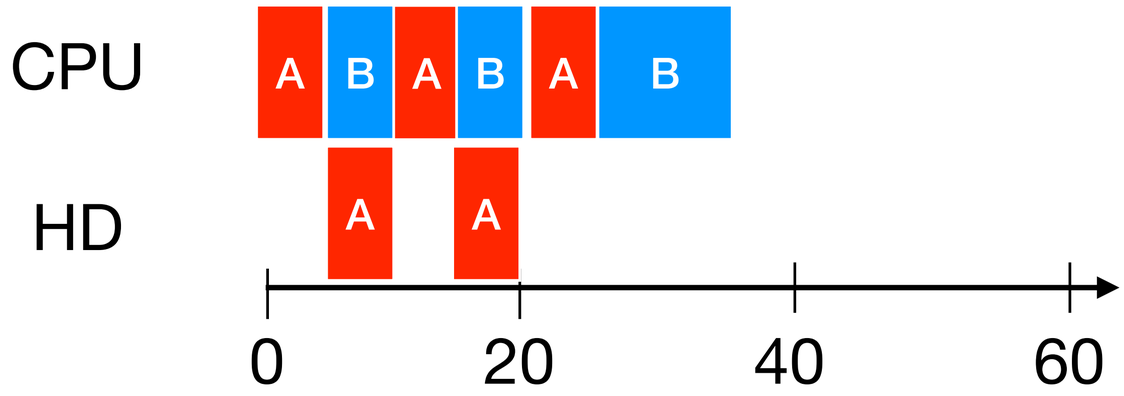
- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - ~~Run to completion~~
  - ~~Run on CPU only (no I/O)~~
  - Have a known run-time

What if processes periodically perform I/O operations?

PSJF + I/O

A=B=25s Arrival: A=B=0





A issues an I/O request every 5s

If A is treated as a single job, CPU is underutilized

It is better to treat each CPU burst as a separate job

Then based on PSJF policy, B will run while A does I/O and will be pre-empted when A returns to CPU

Assumptions

- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - ~~Run to completion~~
  - ~~Run on CPU only (no I/O)~~
  - Have a known run-time

One more assumption left for the next lecture

Summary

- Kernel and user mode
  - User code does not run directly on hardware, only kernel mode does
  - User code calls functions from the OS system call API
  - System calls run in kernel mode and return back to the user
  - Context switch between processes is not free

Summary

- Metrics:
  - **Turnaround** - how quickly the process finishes after arrival
  - **Response time** - how quickly is the process scheduled after arrival
- Scheduling policies
  - Non-preemptive:
    - **FIFO** — simple but ineffective
    - **SJF** — provably optimal turnaround (under unrealistic constraints), poor response time
  - Preemptive
    - **PSJF** — provably optimal turnaround (under unrealistic constraints), poor response time
    - **RR** — better response time at the expense of turnaround

Reading

- Chapter 6 Mechanism: Limited Direct Execution
  - https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-mechanisms.pdf
- Chapter 7 Scheduling: Introduction
  - https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf
- Optional reading
  - Chapter 1.5, *Modern Operating Systems*, Tanenbaum & Bos, 2016

Reminders

- Please post questions on Teams, I get many emails with the same question
- Enable notifications on Teams

Lab 2

- Lab 2 is posted on CourseLink
  - fork(), wait(), exec(), kill() system calls
- You can do it on your own or during the lab session(s) tomorrow

A1 posted

- Due **Feb 1 @ 11:59 PM EST** + 4 grace days
  - Grace days are **per term**, not per assignment
  - Grace days are whole days, 1s late uses 1 day
- Graded out of 100 points:
  - [70 points] - passing autograder tests
  - [10 points] - README.md
  - [up to 20 points] - schedule 100K jobs under 30 s (beat my implementation)

Quizzes

- In-class quizzes are **only** for students who are **present in the classroom**
- **Do not** share links with students who are not in class

Quiz 3

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet